Homework 3: Multi-Agent Search

111550170 卓建均

Part I. Implementation:

Part 1. Minimax Search

The minimax function recursively explores the game tree to determine the best move for Pacman based on the current state and the specified depth. I first check if the depth is zero or if the current state is a winning or losing state. If so, it returns the evaluation of the state. Next, I calculate the next depth and the next agent to consider, cycling through the players (Pacman and Ghosts).
The function then retrieves the legal actions that the current agent can take in the given state.
For each legal action, it generates the next possible states by applying the action to the current state. It recursively calls the minimax function on each of these next states with the updated depth and agent, obtaining their corresponding values. If the current agent is Pacman, it selects the maximum value from next value and the corresponding index. Otherwise, if it's a Ghost, it selects the minimum value. Finally, it returns the best value found and the corresponding legal action at best index, representing the optimal move for Pacman at the current state.

```
def minimax(state, depth, agent):
    if depth == 0 or state.isWin() or state.isLose():
        return self.evaluationFunction(state), None
    nxt_dep = depth
    nxt_agent = (agent + 1) % state.getNumAgents()
    if agent == state.getNumAgents() - 1:
        nxt_dep = depth - 1
    legal_actions = state.getLegalActions(agent)
    nxt_states = [state.getNextState(agent, action) for action in legal_actions]
    nxt_val = [minimax(nxt_state, nxt_dep, nxt_agent)[0] for nxt_state in nxt_states]
    if agent == 0:  # pacman
        best_val = max(nxt_val)
        best_index = nxt_val.index(best_val)
    else:  # ghost
        best_val = min(nxt_val)
        best_index = nxt_val.index(best_val)
    return best_val, legal_actions[best_index]
_, best_action = minimax(gameState, self.depth, 0)
return best_action
```

Part 2. Alpha-Beta Pruning

First, I check if the depth is zero or if the current state is a winning or losing state, if so, it returns the evaluation of the state. It then determines the next agent to move and increments the depth if

all agents have moved. For Pacman, I initialize best score to –infinity and loop through all legal actions for Pacman, calling the function recursively for the next state. It updates best value with the maximum of its current value and the result of the recursive call. If best value becomes greater than beta, it returns best value as the minimizing player will never choose this branch. It also updates alpha with the maximum of its current value and best value. For ghosts, I initialize best value to +infinity. It also loops through all legal actions for the ghost agent, similarly updating best value and beta, by just changing the use of maximum to minimum. If best value becomes less than alpha, it returns best value as the maximizing player will not choose this branch. Then, I set alpha and beta to negative and positive infinity, respectively, initialize max value to negative infinity and best action to the first legal action for Pacman. After that I write a try block which is used to execute the process of evaluating Pacman's best move using the Alpha-Beta Pruning algorithm. It encapsulates the loop that considers all legal actions for Pacman, calculates their utilities, and updates the best action accordingly. The purpose of the try block is to handle any potential exceptions that might occur during this process, ensuring the program continues to run smoothly even if errors arise. Finally, the function returns the best action found, which is the action that leads to the highest utility for Pacman based on the Alpha-Beta Pruning algorithm. This action represents Pacman's best move in the current game state.

```
def alphabeta_prune(agent, depth, gameState, alpha, beta):
    if gameState.isWin() or gameState.isLose() or depth == self.depth:
        return self.evaluationFunction(gameState)
    nxt_agent = agent + 1
    if gameState.getNumAgents() == nxt_agent:
        nxt_agent = 0
        depth += 1
    if agent == 0:  # pacman (max player)
        best_val = float("-inf")
        for action in gameState.getLegalActions(agent):
            best_val = max(best_val, alphabeta_prune(nxt_agent, depth, gameState.getNextState(agent, action), alpha, beta))
            if best_val > beta:
                return best_val
            alpha = max(alpha, best_val)
        return best_val
    else:  # ghost (min player)
        best_val = float("inf")
        for action in gameState.getLegalActions(agent):
            best_val = min(best_val, alphabeta_prune(nxt_agent, depth, gameState.getNextState(agent, action), alpha, beta))
            if best_val < alpha:
                return best_val
            beta = min(beta, best_val)
        return best_val
alpha = float("-inf")
beta = float("inf")
max_val = float("-inf")
best_action = gameState.getLegalActions(0)[0]
```

```
try:
    for action in gameState.getLegalActions(0):
        best_val = alphabeta_prune(1, 0, gameState.getNextState(0, action), alpha, beta)
        if best_val > max_val:
            max_val = best_val
            best_action = action
        alpha = max(alpha, max_val)
except Exception as e:
    print("Exception:", e)
return best_action
```

Part 3. Expectimax Search

First, I check if the depth is 0 or if the current state is in a winning or losing condition. If either condition is met, it returns the evaluation of the state. Next, I calculate the next depth and next agent for the next recursive call and handle if the current agent is the last in the list. Then the function retrieves the legal actions for the current agent, representing the possible actions in the current state. For each legal action, it calculates the next states by applying the action to the state. Next, it recursively calls expectimax function for each next state to calculate the expected values next value. If the current agent is Pacman, it selects the action with the maximum expected value. It finds the best value as the maximum of the next value, then determines the index of the maximum value in the next value. The function returns the best value and the corresponding legal action. If the current agent is not Pacman, it calculates the average of the next value to represent the expected value for that agent's turn. Finally, the function recursively starts with Pacman and determines the best action for Pacman's turn by calling the function. This best action represents Pacman's best move in the current game state, considering the Expectimax search's evaluation of possible outcomes and chance events.

```
def expectimax(state, depth, agent):
    if depth == 0 or state.isWin() or state.isLose():
        return self.evaluationFunction(state), None
    nxt_dep = depth
    nxt_agent = (agent + 1) % state.getNumAgents()
    if agent == state.getNumAgents() - 1:
        nxt_dep = depth - 1
    legal_actions = state.getLegalActions(agent)
    nxt_states = [state.getNextState(agent, action) for action in legal_actions]
    nxt_val = [expectimax(nxt_state, nxt_dep, nxt_agent)[0] for nxt_state in nxt_states]
    if agent == 0:
        best_val = max(nxt_val)
        best_index = nxt_val.index(best_val)
        return best_val, legal_actions[best_index]
    else:
        return sum(nxt_val)/len(nxt_val), 0
_, best_action = expectimax(gameState, self.depth, 0)
return best_action
```

Part 4. Evaluation Function

I consider several factors, including the current score, Pacman's position, the positions of food pellets and capsules, as well as information about ghosts and weights assigned to these factors. The score is computed based on these considerations to aid Pacman in making decisions.
For score calculation, I set the initial score to the current game score, and store Pacman's current position in ‘position’, remaining food pellets in 'food', capsule positions in 'capsule' and ghost states in 'ghost state'. For the new weight, I set food weight to 10, capsule weight to 50,

ghost weight to -30 and scared ghost weight to 300. Next, I calculate the closest capsule distance by figuring out how close Pacman is to the closest capsule, and then adds the value to Pacman's score depending on this distance. If no capsule is nearby, add the food weight to the score. Similarly, I use the same way to calculate closest food distance by figuring out how close Pacman is to the closest food, and then adds the value to Pacman's score depending on this distance. If no food is nearby, add the food weight to the score. For the ghosts, I calculate Pacman's distance to each ghost. If a ghost isn't at Pacman's position and the ghost is scared, add scared ghost weight divided by the distance to the score. If the ghost isn't scared, add ghost weight divided by the distance to the score. Finally, it returns the final score.

```
score = currentGameState.getScore()
position = currentGameState.getPacmanPosition()
food = currentGameState.getFood()
capsule = currentGameState.getCapsules()
ghost_state = currentGameState.getGhostStates()
food_weight = 10
capsule_weight = 50
ghost_weight = -30
scared_ghost_weight = 300
closest_capsuleDist = float("inf")
for capsulePosition in capsule:
    capsuledist = manhattanDistance(position, capsulePosition)
    closest_capsuleDist = min(closest_capsuleDist, capsuledist)
if closest_capsuleDist  < float("inf"):
    score += capsule_weight / closest_capsuleDist
else:
    score += food_weight
closest_foodDist = float("inf")
for foodPosition in food.asList():
    fooddist = manhattanDistance(position, foodPosition)
    closest_foodDist = min(closest_foodDist, fooddist)
if closest_foodDist < float("inf"):
    score += food_weight / closest_foodDist
else:
    score += food_weight
for ghost in ghost_state:
    dist = manhattanDistance(position, ghost.getPosition())
    if dist > 0:
        if ghost.scaredTimer > 0:
            score += scared_ghost_weight / dist
        else:
            score += ghost_weight / dist
return score
```

Part II. Results & Analysis:

Part 1. Overall Result

```
Finished at 17:52:42

Provisional grades
==================
Question part1: 15/15
Question part2: 20/20
Question part3: 20/20
Question part4: 10/10
------------------
Total: 65/65
```

Part 1. Minimax Search

```
Question part1
==============

*** PASS: test_cases\part1\0-eval-function-lose-states-1.test
*** PASS: test_cases\part1\0-eval-function-lose-states-2.test
*** PASS: test_cases\part1\0-eval-function-win-states-1.test
*** PASS: test_cases\part1\0-eval-function-win-states-2.test
*** PASS: test_cases\part1\0-lecture-6-tree.test
*** PASS: test_cases\part1\0-small-tree.test
*** PASS: test_cases\part1\1-1-minmax.test
*** PASS: test_cases\part1\1-2-minmax.test
*** PASS: test_cases\part1\1-3-minmax.test
*** PASS: test_cases\part1\1-4-minmax.test
*** PASS: test_cases\part1\1-5-minmax.test
*** PASS: test_cases\part1\1-6-minmax.test
*** PASS: test_cases\part1\1-7-minmax.test
*** PASS: test_cases\part1\1-8-minmax.test
*** PASS: test_cases\part1\2-1a-vary-depth.test
*** PASS: test_cases\part1\2-1b-vary-depth.test
*** PASS: test_cases\part1\2-2a-vary-depth.test
*** PASS: test_cases\part1\2-2b-vary-depth.test
*** PASS: test_cases\part1\2-3a-vary-depth.test
*** PASS: test_cases\part1\2-3b-vary-depth.test
*** PASS: test_cases\part1\2-4a-vary-depth.test
*** PASS: test_cases\part1\2-4b-vary-depth.test
*** PASS: test_cases\part1\2-one-ghost-3level.test
*** PASS: test_cases\part1\3-one-ghost-4level.test
*** PASS: test_cases\part1\4-two-ghosts-3level.test
*** PASS: test_cases\part1\5-two-ghosts-4level.test
*** PASS: test_cases\part1\6-tied-root.test
*** PASS: test_cases\part1\7-1a-check-depth-one-ghost.test
*** PASS: test_cases\part1\7-1b-check-depth-one-ghost.test
*** PASS: test_cases\part1\7-1c-check-depth-one-ghost.test
*** PASS: test_cases\part1\7-2a-check-depth-two-ghosts.test
*** PASS: test_cases\part1\7-2b-check-depth-two-ghosts.test
*** PASS: test_cases\part1\7-2c-check-depth-two-ghosts.test
*** Running MinimaxAgent on smallClassic 1 time(s).
Pacman died! Score: 84
Average Score: 84.0
Scores:        84.0
Win Rate:      0/1 (0.00)
Record:        Loss
*** Finished running MinimaxAgent on smallClassic after 0 seconds.
*** Won 0 out of 1 games. Average score: 84.000000 ***
*** PASS: test_cases\part1\8-pacman-game.test

### Question part1: 15/15 ###
```

Part 2. Alpha-Beta Pruning

```
Question part2
==============

*** PASS: test_cases\part2\0-eval-function-lose-states-1.test
*** PASS: test_cases\part2\0-eval-function-lose-states-2.test
*** PASS: test_cases\part2\0-eval-function-win-states-1.test
*** PASS: test_cases\part2\0-eval-function-win-states-2.test
*** PASS: test_cases\part2\0-lecture-6-tree.test
*** PASS: test_cases\part2\0-small-tree.test
*** PASS: test_cases\part2\1-1-minmax.test
*** PASS: test_cases\part2\1-2-minmax.test
*** PASS: test_cases\part2\1-3-minmax.test
*** PASS: test_cases\part2\1-4-minmax.test
*** PASS: test_cases\part2\1-5-minmax.test
*** PASS: test_cases\part2\1-6-minmax.test
*** PASS: test_cases\part2\1-7-minmax.test
*** PASS: test_cases\part2\1-8-minmax.test
*** PASS: test_cases\part2\2-1a-vary-depth.test
*** PASS: test_cases\part2\2-1b-vary-depth.test
*** PASS: test_cases\part2\2-2a-vary-depth.test
*** PASS: test_cases\part2\2-2b-vary-depth.test
*** PASS: test_cases\part2\2-3a-vary-depth.test
*** PASS: test_cases\part2\2-3b-vary-depth.test
*** PASS: test_cases\part2\2-4a-vary-depth.test
*** PASS: test_cases\part2\2-4b-vary-depth.test
*** PASS: test_cases\part2\2-one-ghost-3level.test
*** PASS: test_cases\part2\3-one-ghost-4level.test
*** PASS: test_cases\part2\4-two-ghosts-3level.test
*** PASS: test_cases\part2\5-two-ghosts-4level.test
*** PASS: test_cases\part2\6-tied-root.test
*** PASS: test_cases\part2\7-1a-check-depth-one-ghost.test
*** PASS: test_cases\part2\7-1b-check-depth-one-ghost.test
*** PASS: test_cases\part2\7-1c-check-depth-one-ghost.test
*** PASS: test_cases\part2\7-2a-check-depth-two-ghosts.test
*** PASS: test_cases\part2\7-2b-check-depth-two-ghosts.test
*** PASS: test_cases\part2\7-2c-check-depth-two-ghosts.test
*** Running AlphaBetaAgent on smallClassic 1 time(s).
```

```
Pacman died! Score: 84
Average Score: 84.0
Scores:        84.0
Win Rate:      0/1 (0.00)
Record:        Loss
*** Finished running AlphaBetaAgent on smallClassic after 0 seconds.
*** Won 0 out of 1 games. Average score: 84.000000 ***
*** PASS: test_cases\part2\8-pacman-game.test

### Question part2: 20/20 ###
```

Part 3. Expectimax Search

```
Question part3
==============

*** PASS: test_cases\part3\0-eval-function-lose-states-1.test
*** PASS: test_cases\part3\0-eval-function-lose-states-2.test
*** PASS: test_cases\part3\0-eval-function-win-states-1.test
*** PASS: test_cases\part3\0-eval-function-win-states-2.test
*** PASS: test_cases\part3\0-expectimax1.test
*** PASS: test_cases\part3\1-expectimax2.test
*** PASS: test_cases\part3\2-one-ghost-3level.test
*** PASS: test_cases\part3\3-one-ghost-4level.test
*** PASS: test_cases\part3\4-two-ghosts-3level.test
*** PASS: test_cases\part3\5-two-ghosts-4level.test
*** PASS: test_cases\part3\6-1a-check-depth-one-ghost.test
*** PASS: test_cases\part3\6-1b-check-depth-one-ghost.test
*** PASS: test_cases\part3\6-1c-check-depth-one-ghost.test
*** PASS: test_cases\part3\6-2a-check-depth-two-ghosts.test
*** PASS: test_cases\part3\6-2b-check-depth-two-ghosts.test
*** PASS: test_cases\part3\6-2c-check-depth-two-ghosts.test
*** Running ExpectimaxAgent on smallClassic 1 time(s).
Pacman died! Score: 84
Average Score: 84.0
Scores:        84.0
Win Rate:      0/1 (0.00)
Record:        Loss
*** Finished running ExpectimaxAgent on smallClassic after 0 seconds.
*** Won 0 out of 1 games. Average score: 84.000000 ***
*** PASS: test_cases\part3\7-pacman-game.test

### Question part3: 20/20 ###
```

Part 4. Evaluation Function

```
Question part4
==============

Pacman emerges victorious! Score: 1145
Pacman emerges victorious! Score: 1345
Pacman emerges victorious! Score: 1340
Pacman emerges victorious! Score: 1324
Pacman emerges victorious! Score: 1298
Pacman emerges victorious! Score: 1329
Pacman emerges victorious! Score: 1179
Pacman emerges victorious! Score: 1338
Pacman emerges victorious! Score: 1192
Pacman emerges victorious! Score: 1294
Average Score: 1278.4
Scores:        1145.0, 1345.0, 1340.0, 1324.0, 1298.0, 1329.0, 1179.0, 1338.0, 1192.0, 1294.0
Win Rate:      10/10 (1.00)
Record:        Win, Win, Win, Win, Win, Win, Win, Win, Win, Win
*** PASS: test_cases\part4\grade-agent.test (8 of 8 points)
*** EXTRA CREDIT: 2 points
***     1278.4 average score (4 of 4 points)
***         Grading scheme:
***          < 600:  0 points
***         >= 600:  2 points
***         >= 1200:  4 points
***     10 games not timed out (2 of 2 points)
***         Grading scheme:
***          < 0:  fail
***         >= 0:  0 points
***         >= 5:  1 points
***         >= 10:  2 points
***     10 wins (4 of 4 points)
***         Grading scheme:
***          < 1:  fail
***         >= 1:  1 points
***         >= 4:  2 points
***         >= 7:  3 points
***         >= 10:  4 points

### Question part4: 10/10 ###
```